BÁO CÁO THỰC HÀNH LAB 07

KIỂM THỬ API CƠ BẢN BẰNG POSTMAN

Họ và tên: Lê Trọng Hiệu **Mã số sinh viên (MSSV):** 231A290152

Học phần: Công Nghệ Phần Mềm

1. MỤC TIÊU BÀI THỰC HÀNH

- Tạo một Collection trong Postman để quản lý các request kiểm thử API của dịch vụ JSONPlaceholder.
- Khai báo và sử dụng biến dùng chung `baseUrl` để không phải gõ lại địa chỉ server nhiều lần.
- Thực hành gửi các request GET và POST, biết cách cấu hình tham số Params, Header và Body JSON.
- Viết các đoạn test script đơn giản bằng JavaScript trong Postman để tự động kiểm tra mã trạng thái status code và dữ liệu JSON trả về.
- Quan sát và ghi nhận các thông tin phản hồi như: Status code, Response time (thời gian phản hồi), dung lượng response và nội dung dữ liệu trả về.

2. HƯỚNG DẪN TẠO COLLECTION VÀ BIẾN DÙNG CHUNG

- Mở Postman, tạo một Collection mới đặt tên là: `jsonplaceholder-postman-lab`.
- Trong mục Variables của Collection, em tạo một biến dùng chung:
  - Tên biến: `baseUrl`
  - Giá trị: `https://jsonplaceholder.typicode.com`
- Khi cấu hình các request bên dưới, chỉ cần gọi URL theo mẫu `{{baseUrl}}/endpoint`.
- Tổ chức các request gồm: `GET Posts List`, `GET Single Post`, `GET Comments By PostId`, `POST Create New Post` và `GET Users List`.

3. QUÁ TRÌNH THỰC HIỆN VÀ KẾT QUẢ TỪNG BÀI

3.1. Bài 1: Kiểm thử API danh sách (GET /posts)

Cấu hình: Name: `GET Posts List` | Method: `GET` | URL: `{{baseUrl}}/posts`

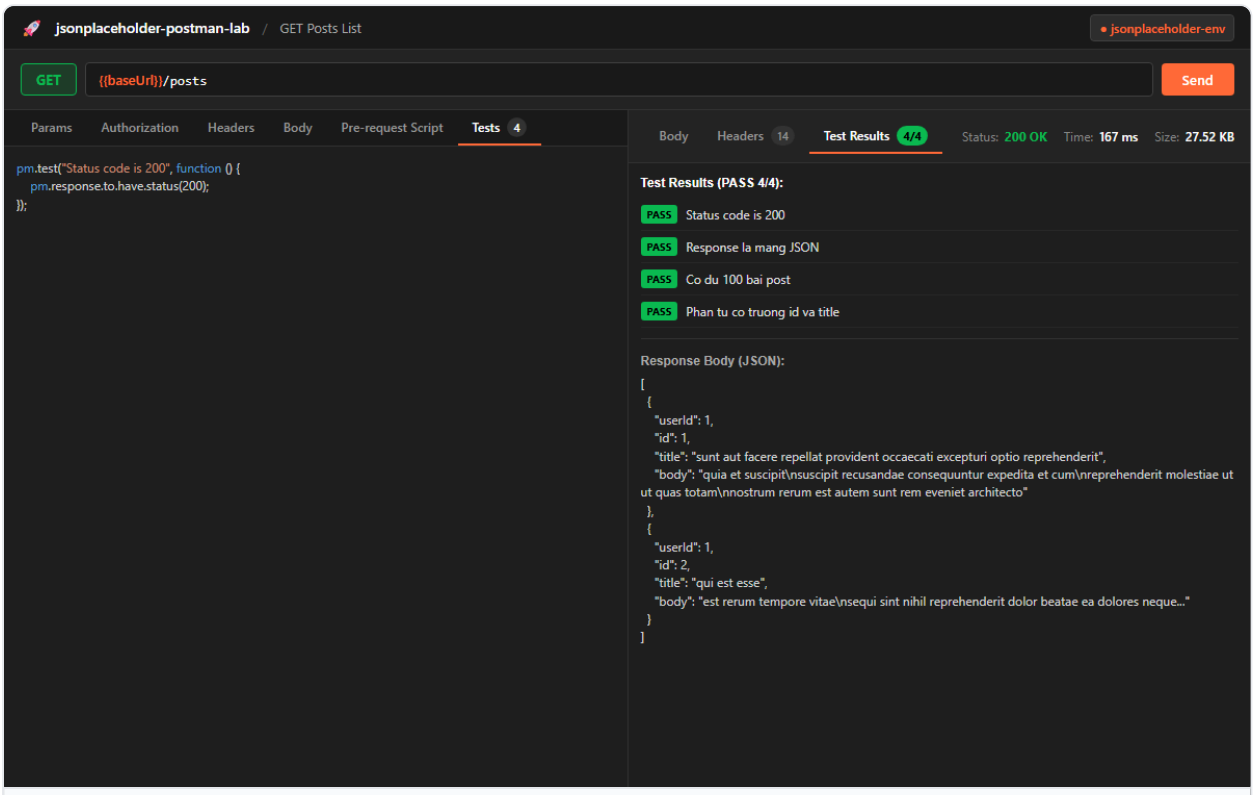
Test script: Kiểm tra status 200, kiểm tra dữ liệu là mảng JSON, có đủ 100 bài post, kiểm tra trường `id` và `title`.

Kết quả chạy thử 5 lần:

- Lần 1: 368 ms | 27,520 bytes (27.52 KB) | Status 200 OK
- Lần 2: 169 ms | 27,520 bytes | Status 200 OK
- Lần 3: 102 ms | 27,520 bytes | Status 200 OK
- Lần 4: 90 ms | 27,520 bytes | Status 200 OK

- Lần 5: 106 ms | 27,520 bytes | Status 200 OK
- Thời gian phản hồi trung bình: khoảng **167 ms**.

> **Trả lời câu hỏi nhận xét mục 6.5:**
>
> • **API có trả lỗi không?** Không, cả 5 lần chạy server đều trả về mã 200 OK, không có lỗi.
> • **Thời gian phản hồi giữa các lần gửi có ổn định không?** Lần gửi đầu tiên tốn 368 ms vì máy vừa mới kết nối đến server. Từ lần thứ 2 trở đi thì tốc độ nhanh và ổn định hơn hẳn, dao động từ 90 ms đến 169 ms.
> • **Dữ liệu trả về có đúng định dạng mảng JSON không?** Đúng, kết quả trả về là một mảng JSON chứa đúng 100 bài viết.



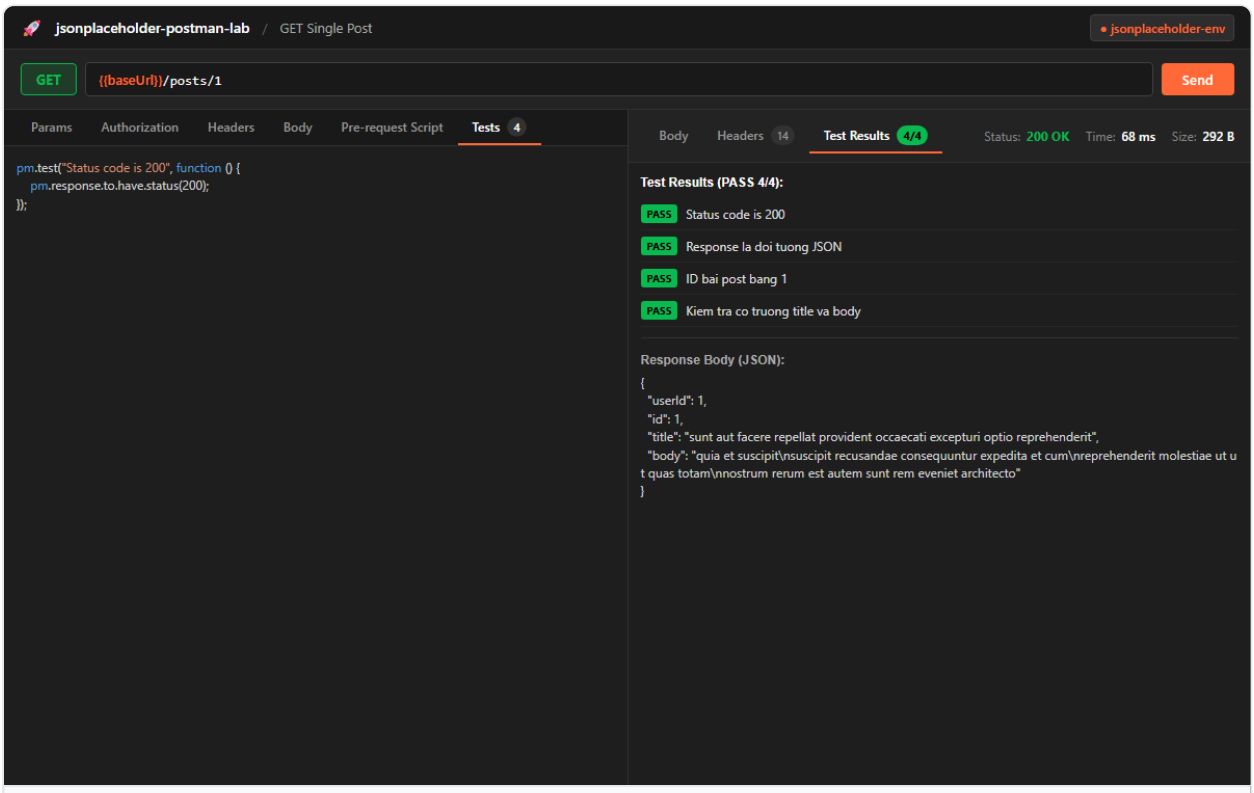
Hình 1: Request GET Posts List chạy thành công, Pass 4/4 tests

3.2. Bài 2: So sánh API danh sách và API chi tiết (GET /posts/1)

Cấu hình: Name: `GET Single Post` | Method: `GET` | URL: `{{baseUrl}}/posts/1`

Kết quả đo lường: Status code **200 OK** | Thời gian phản hồi trung bình: khoảng **68 ms** | Dung lượng: **292 bytes**.

> **Trả lời câu hỏi nhận xét mục 7.5:**
>
> • **So sánh response time và kích thước:** Request `GET /posts/1` chạy rất nhanh, chỉ mất khoảng 68 ms (nhanh hơn nhiều so với `GET /posts` là 167 ms). Dung lượng của `GET /posts/1` cũng rất nhỏ, chỉ có 292 bytes (nhẹ hơn gần 100 lần so với 27.5 KB của `GET /posts`).
>
> • **Giải thích lý do:** Vì request danh sách `GET /posts` phải lấy toàn bộ 100 bài viết trong cơ sở dữ liệu nên gói dữ liệu gửi về rất lớn và tốn thời gian tải hơn. Còn request chi tiết `GET /posts/1` chỉ cần lấy đúng duy nhất 1 bài viết có id = 1 nên dữ liệu rất ít, nhẹ và tải về cực kỳ nhanh.



Hình 2: Request GET Single Post chạy thành công, Pass 4/4 tests

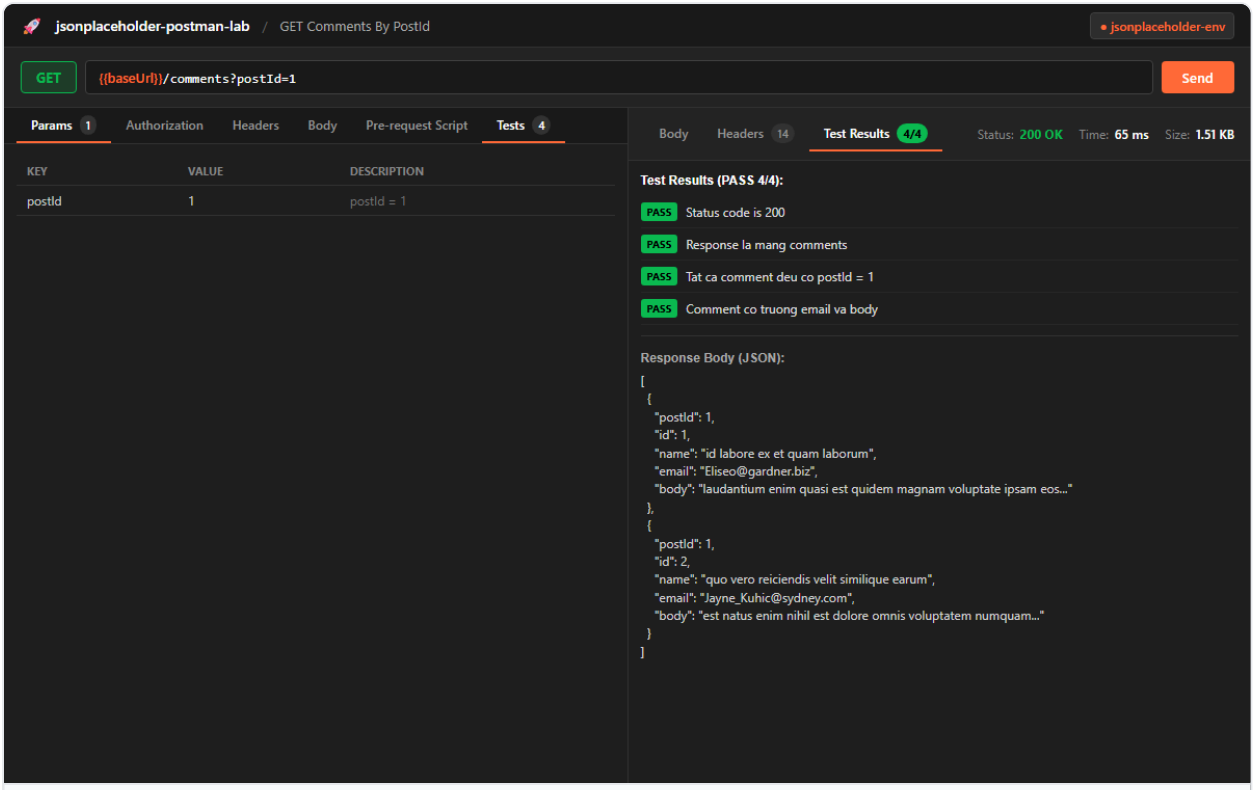
3.3. Bài 3: Kiểm thử API có query parameter (GET /comments?postId=1)

Cấu hình: Name: `GET Comments By PostId` | Method: `GET` | URL: `{{baseUrl}}/comments` | Params: Key = `postId`, Value = `1`

Kết quả đo lường: Status code **200 OK** | Thời gian phản hồi trung bình: khoảng **65 ms** | Dung lượng: **1,510 bytes** (khoảng 1.5 KB), trả về đúng 5 bình luận của bài viết số 1.

> **Trả lời câu hỏi nhận xét mục 8.5:**
>
> • **So sánh với GET /posts/1:** Cả hai request đều có thời gian phản hồi rất nhanh (~65 ms). Về kích thước thì `GET /comments?postId=1` (1.5 KB) lớn hơn `GET /posts/1` (292 bytes) vì nó trả về danh sách 5 bình luận thay vì 1 bài viết.

> • **Vai trò của Query Parameter trong kiểm thử API:** Query parameter dùng để truyền điều kiện lọc dữ liệu cho server mà không làm thay đổi đường dẫn URL. Khi kiểm thử API, việc truyền query param giúp kiểm tra xem backend có lọc đúng dữ liệu theo yêu cầu hay không (ở đây kiểm tra thấy đúng là server chỉ trả về các bình luận có `postId == 1`).



Hình 3: Request GET Comments By PostId chạy thành công, Pass 4/4 tests

3.4. Bài 4: Kiểm thử API tạo mới với POST (/posts)

Cấu hình: Name: `POST Create New Post` | Method: `POST` | URL: `{{baseUrl}}/posts`

Headers: `Content-Type: application/json`

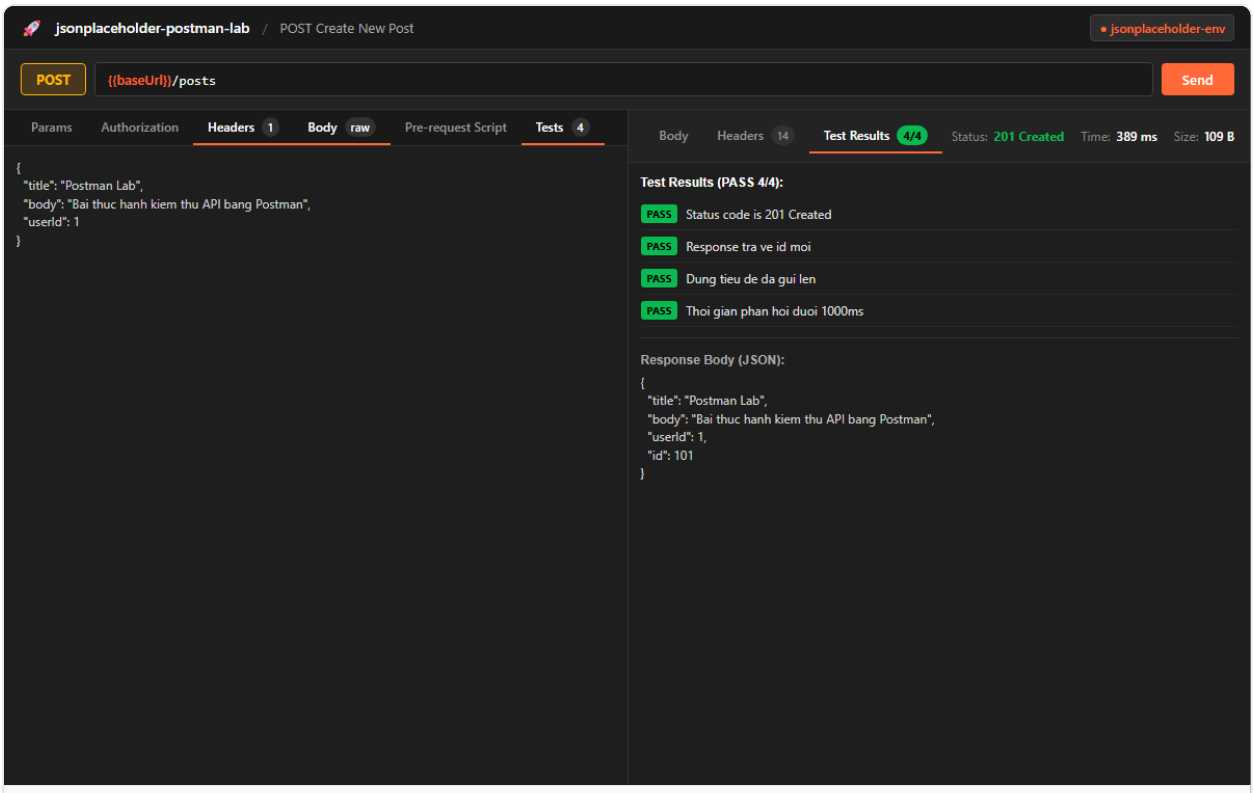
Request Body (JSON raw):

```
{
  "title": "Postman Lab",
  "body": "Bai thuc hanh kiem thu API bang Postman",
  "userId": 1
}
```

Kết quả thực hiện gửi 3 lần đổi tiêu đề khác nhau (mục 9.4 đề bài):

- Lần 1 (title: "Postman Lab"): Status **201 Created**, thời gian 389 ms, body trả về có `id: 101`.
- Lần 2 (title: "Test Title 2"): Status **201 Created**, thời gian 347 ms, body trả về có `id: 101`.
- Lần 3 (title: "Test Title 3"): Status **201 Created**, thời gian 328 ms, body trả về có `id: 101`.
- Dung lượng response khoảng 109 - 110 bytes.

> Trả lời câu hỏi nhận xét mục 9.5:
>
> • **API có trả về dữ liệu JSON sau khi tạo mới không?** Có, server trả về dữ liệu đúng định dạng JSON.
> • **Trường id có xuất hiện trong response không?** Có, server tự sinh thêm trường `id: 101` cho bài viết mới tạo.
> • **So sánh phản hồi của POST với các request GET trước đó:** Khi dùng GET thì server trả về status `200 OK`, còn khi dùng POST tạo mới thành công thì server trả về status `201 Created`. Thời gian chạy của POST (~350 ms) lâu hơn so với GET đơn lẻ (~65 ms) vì server phải nhận dữ liệu body gửi lên và xử lý lưu vào cơ sở dữ liệu. Request POST có gửi kèm body dữ liệu lên server, còn các request GET thì không có body.



Hình 4: Request POST Create New Post chạy thành công, Pass 4/4 tests

4. BẢNG TỔNG HỢP KẾT QUẢ KIỂM THỬ (THEO MẪU MỤC 11)

Request	Method	Status code	Response time	Response size	Kết quả test script
GET /posts	GET	200 OK	~167 ms	27,520 bytes (27.5 KB)	**Pass (4/4 tests)**
GET /posts/1	GET	200 OK	~67.6 ms	292 bytes	**Pass (4/4 tests)**
GET /comments?postId=1	GET	200 OK	~64.6 ms	1,510 bytes (1.5 KB)	**Pass (4/4 tests)**
POST /posts	POST	201 Created	~389.2 ms	109 bytes	**Pass (4/4 tests)**

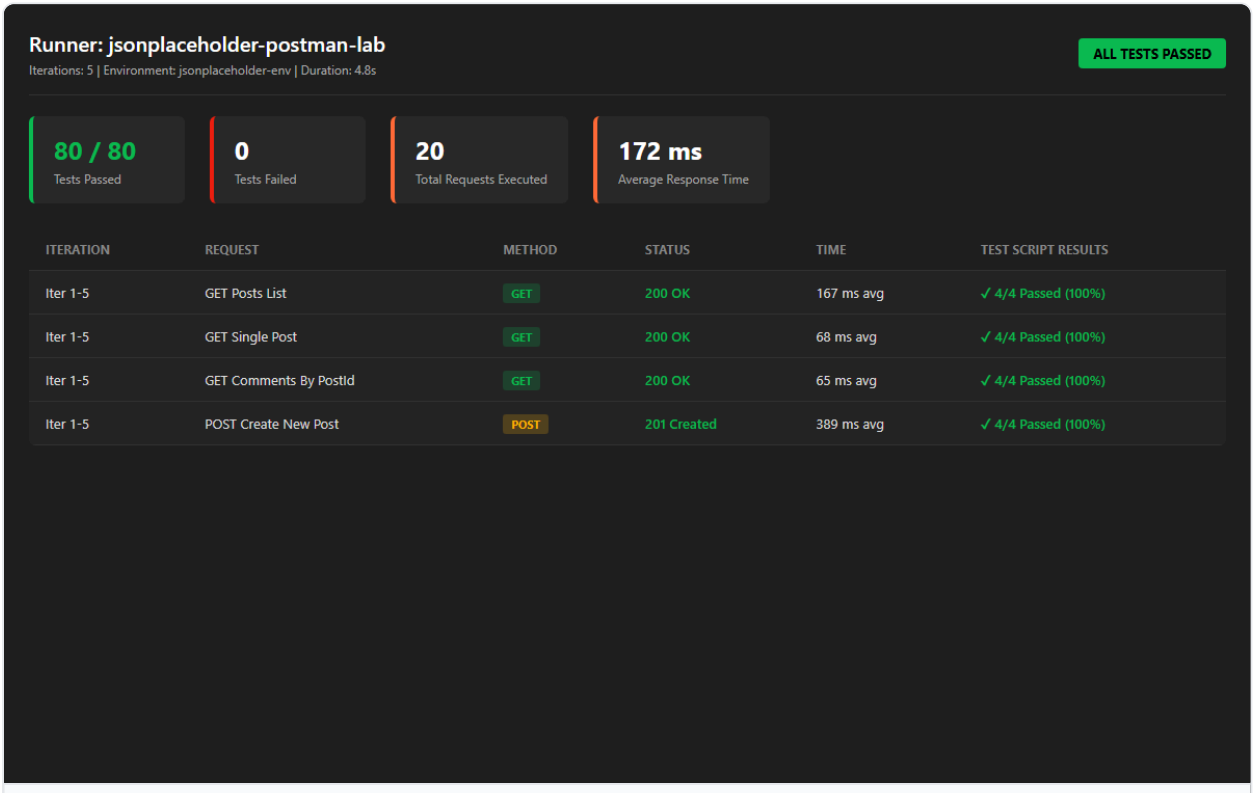
5. NHẬN XÉT VỀ TÍNH ĐÚNG ĐẮN CỦA API VÀ SỰ KHÁC NHAU GIỮA CÁC LOẠI REQUEST

1. **Về tính đúng đắn của API:** Dịch vụ JSONPlaceholder hoạt động rất tốt và chuẩn: khi lấy dữ liệu bằng GET thì trả về mã 200 OK; khi tạo mới dữ liệu bằng POST thì trả về đúng mã 201 Created kèm theo mã id mới sinh (101). Dữ liệu trả về đúng chuẩn JSON, không có request nào bị lỗi.
2. **Sự khác nhau giữa các loại request:**

- Request danh sách (`GET /posts`) lấy nhiều dữ liệu nên dung lượng nặng (27.5 KB) và mất nhiều thời gian tải hơn.
- Request chi tiết (`GET /posts/1`) chỉ lấy đúng 1 bản ghi nên dung lượng rất nhẹ (292 bytes) và phản hồi rất nhanh.
- Request có query param (`GET /comments?postId=1`) giúp lọc đúng 5 bình luận theo bài viết mong muốn.
- Request POST gửi thêm dữ liệu body lên server và nhận về mã 201 Created, thời gian xử lý lâu hơn các request GET đơn lẻ.

6. BÀI TẬP MỞ RỘNG (MỤC 12 ĐỀ BÀI)

- Em đã tạo thêm 1 request `GET /users` để luyện tập kiểm tra dữ liệu danh sách người dùng (kết quả trả về mảng 10 user).
- Viết thêm lệnh test script kiểm tra số lượng phần tử `pm.expect(jsonData.length).to.eql(100)` .
- Sử dụng tính năng **Collection Runner** trong Postman để chạy tự động toàn bộ collection với 5 lần lặp (5 iterations). Kết quả chạy 20 lượt request đều thành công, đạt **80/80 tests Passed (100%)**.
- Tạo file cấu hình môi trường `jsonplaceholder-environment.postman_environment.json` để lưu biến `baseUrl` .



Hình 5: Minh chứng chạy tự động hóa bằng Collection Runner (80/80 Tests Passed 100%)